ASSIGNMENT REPORT

Inter-Process Communication

A client and a server program in Python using sockets

Parallel and Distributed Systems

Session 3

Written by

Mohammad Istahad Alam Chowdhury

NIM 20250040225

Class TI25I

Informatics Engineering

Universitas Nusa Putra

Lecturer: Muchtar Ali Setyo Yudono, S.T., M.T.

22 October 2026

1. Introduction

A distributed system is made of programs that run separately and still need to exchange data. Before any two machines can work together they need a channel to send bytes through, and sockets are the usual way to build that channel in Python. A socket is just an address (IP address plus port number) that one program can listen on and another program can connect to.

This assignment asks for two programs. The server listens on a port, waits for a client, prints whatever the client sends, and answers with a confirmation before closing the connection and waiting for the next client. The client connects to that server and lets the user type messages one after another, printing the reply each time. Both programs only use the standard library, so there was nothing to install.

I wrote and tested everything on Windows with Python 3.13.5. All the screenshots and numbers in this report come from real runs of the two programs.

2. How the two programs talk to each other

The server creates a socket, binds it to port 5000, and calls listen(). From then on the operating system queues any incoming connection until the program calls accept(). When a client calls connect(), the operating system completes a TCP handshake and accept() returns a new socket that belongs only to that client. The original listening socket stays open, which is why the server can carry on serving clients after the first one leaves.

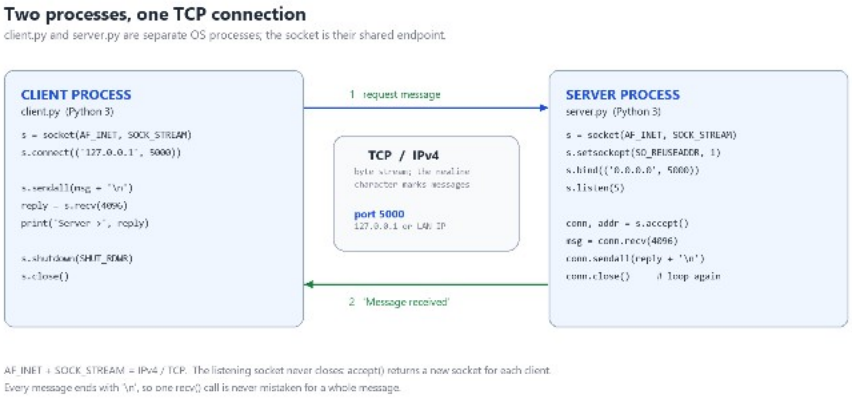


Figure 1. The client and the server are separate processes joined by one TCP connection on port 5000.

TCP gives a stream of bytes, not a stream of messages. If the client sends three messages quickly, the server may receive the first in two pieces or the first two messages in one read. To keep messages apart both programs end every message with a newline character and keep reading until that newline arrives. Figure 2 shows one full exchange between the two programs.

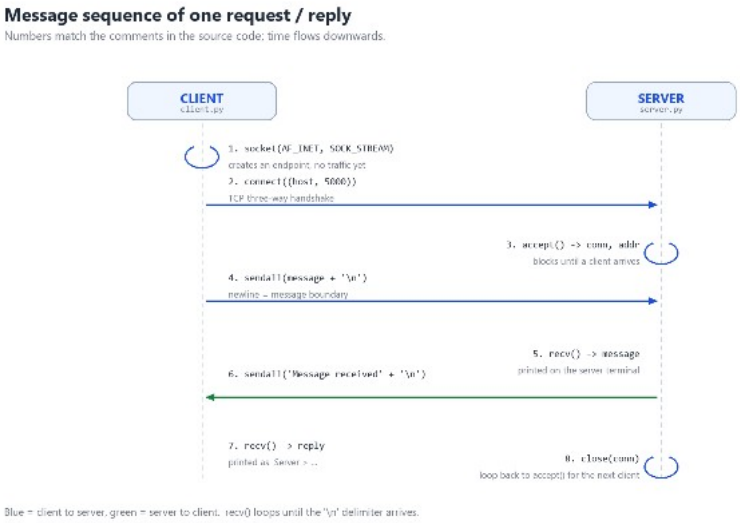


Figure 2. The calls made during one message and its confirmation.

3. The server program (Task 1)

The main loop of server.py accepts one client at a time, which is what the assignment asks for. Each connection gets its own socket, so closing it after the reply does not disturb the listening socket. The reply also tells the client which message it was.

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((host, port))
server.listen(BACKLOG)

while True:
    conn, addr = server.accept()
    with conn:
        handle_client(conn, addr, client_no, timeout, max_messages)
```

For every message the server prints the text, the sender's address, the length in characters and bytes, and the time it arrived. It then sends back "Message received (43 chars / 43 bytes) - thank you, client #1" and closes the connection. A timeout of 30 seconds makes sure a client that connects and then stays silent cannot block everyone behind it.

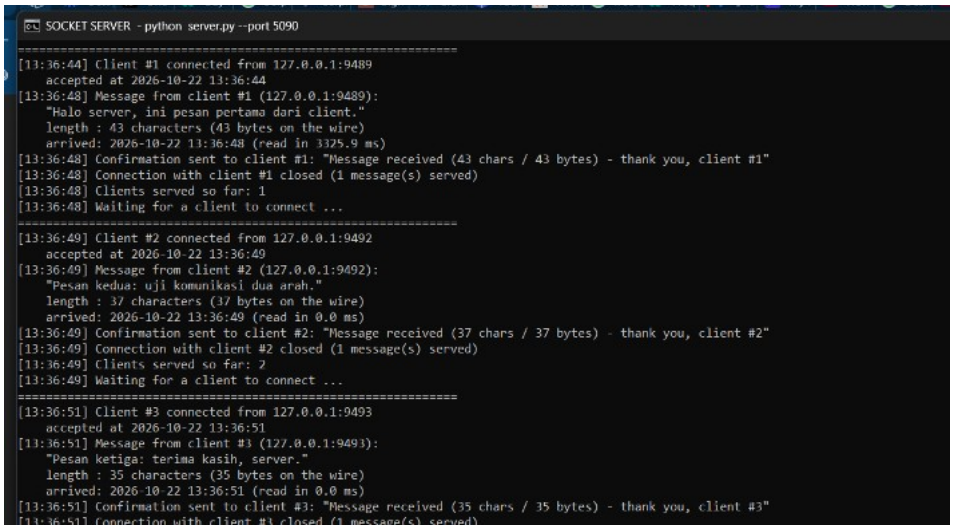


Figure 3. The server terminal after three clients were served one after another, each with its own number.

4. The client program (Task 2)

The client connects once and then loops on input(), so the user can keep sending messages without restarting the program. For each line it sends the text, waits for the answer, and prints the round trip time measured with time.perf_counter().

```
while True:
    message = input("\nYou > ").strip()
    if not message:
        continue
    if message.lower() in QUIT_COMMANDS:      # /quit, /exit, :q
        break
    sock, reply, reconnected = exchange(host, port, sock, message)
    print(f"Server > {reply}")
```

One thing needed extra work. The server closes the connection after every reply, so the second message would be sent to a socket that no longer exists. The client therefore checks whether the reply came back cleanly, and if the connection is gone it opens a new one and sends the message again. The user does not notice anything except a short note in the output.

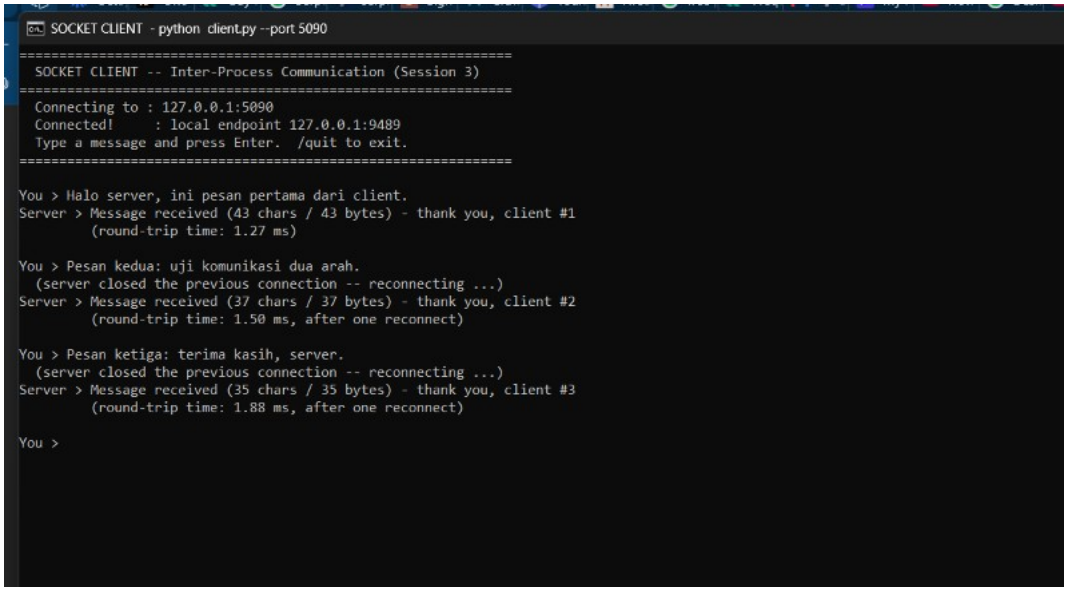


Figure 4. Three messages typed in the client, with the reply and the round trip time for each.

5. How to run the two programs

Both files sit in the same folder. The server has to be started first, in its own terminal window; the client then goes in a second window. If the client is started first it prints a message saying the connection was refused.

```
python server.py              (first window)
python client.py              (second window, same port)
```

On Windows the folder also contains start_demo.bat, which opens both windows with the correct port already filled in. In the client window every line typed and confirmed with Enter is sent to the server, and the answer appears underneath it. Typing /quit closes the connection and the server keeps waiting for the next client until Ctrl+C is pressed.

The server listens on port 5000 and accepts other machines on the same network, so the client can be pointed at the server's address with python client.py --host 192.168.1.10. It is worth saying that opening the port in a web browser does not work and is not supposed to: the browser would send an HTTP

request, while this server only understands the newline protocol from section 2. The client program is the only thing that can talk to it.

6. Testing

I tested with a script that starts the real server on a free port, sends real messages over a real socket, and compares the answers. That way the tests do not depend on me typing correctly. All seven cases pass.

#	What was tested	Expected	Result
1	One client sends one message	Reply starts with "Message received"	Pass
2	Five clients one after another	Each served, numbered in order	Pass
3	Text with accents and an emoji	45 characters counted correctly	Pass
4	A message of 5000 characters	Received in full, not cut short	Pass
5	A client that connects and leaves	Server keeps working	Pass
6	An empty message	No reply, connection closed	Pass
7	Four clients at the same time (bonus server)	All four answered in 0.50 s	Pass

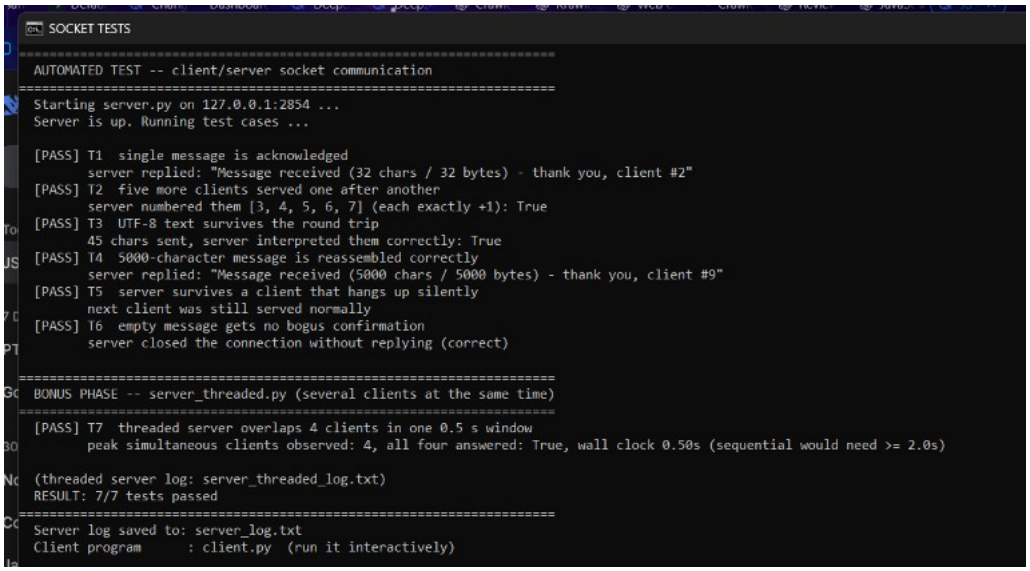


Figure 5. The test script reporting 7 of 7 cases passed.

7. Problems I ran into

Three problems were not obvious at first and each one only showed up because I ran the programs instead of only reading the code.

The first was WinError 10053. My first client worked once and then crashed on the second message, because the server had already closed the connection. The fix is the reconnect logic in section 4.

The second was an encoding error. The server stopped completely when a message contained an emoji, with UnicodeEncodeError from the charmap codec. The network part was fine; the problem was that a

Windows console still uses an old code page by default. Setting the standard output to UTF-8 fixed it, and the same change on the input side stopped accented letters from arriving garbled.

The third was a truncated message. A single recv() call is not guaranteed to return a whole message, and in testing a long message arrived in several pieces. Reading in a loop until the newline turned up fixed it, which is also why test 4 exists.

Apart from those, I added two small things while testing: SO_REUSEADDR, because restarting the server sometimes failed with "address already in use", and sendall() instead of send(), because send() can accept only part of a long message.

I also wrote a second version of the server that handles each client in its own thread, as an extra. With four clients that each kept the connection open for half a second, the threaded server finished in 0.50 seconds. A sequential server cannot overlap them, so it would need at least two seconds for the same work.

8. Conclusion

The two programs do what the assignment asked. The server accepts clients one at a time, prints every message it receives, answers with a confirmation and closes the connection properly. The client connects, sends typed messages in a loop and shows the replies with their round trip times. Seven automated tests pass, including a 5000-character message and text with non-ASCII characters.

The part I did not expect was how much of the work is deciding where a message ends and what to do when the other side hangs up. TCP only delivers bytes, so the newline rule had to be written by hand, and the server closing after each reply forced the client to reconnect. Testing each of those cases was what turned the assignment from something that looked right into something that actually works.

References

[1] Python Software Foundation. "socket â€” Low-level networking interface." Python 3 documentation. https://docs.python.org/3/library/socket.html

[2] Kurose, J. F. and Ross, K. W. Computer Networking: A Top-Down Approach, 8th edition. Pearson, 2021.

[3] Tanenbaum, A. S. and van Steen, M. Distributed Systems, 4th edition, version 4.02. 2023.

[4] Stevens, W. R., Fenner, B. and Rudoff, A. M. UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd edition. Addison-Wesley, 2003.

[5] RFC 9293. Transmission Control Protocol (TCP). IETF, 2022.